

Industrial Tool

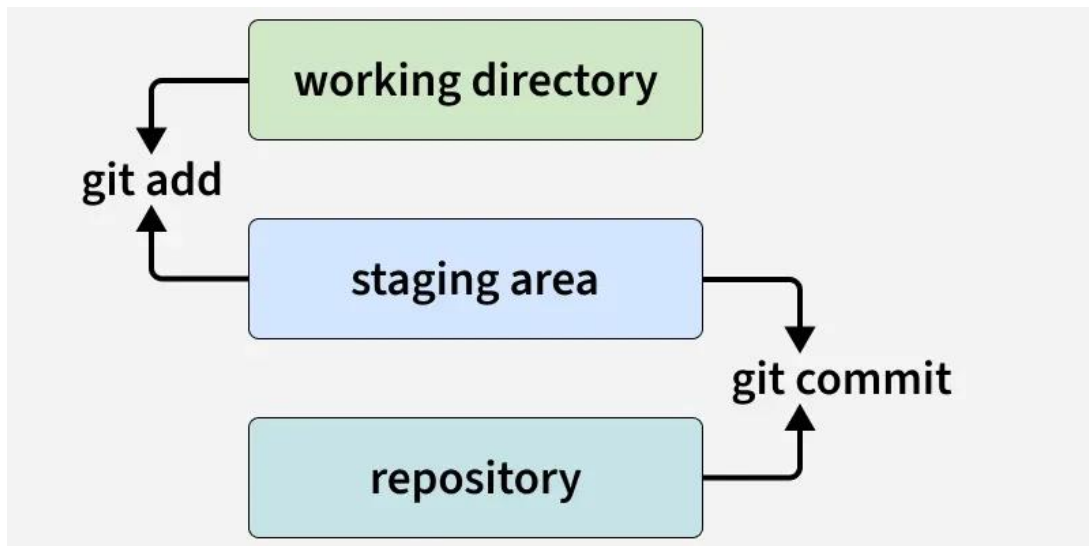
Git & Github

Staging in Git :-

- The Git staging area (index) is a space to gather changes before committing.
- It lets you organize and review what to include in the next commit.

The staging area allows you to:

- Selectively choose changes to commit.
- Break down large changes into smaller, logical commits.
- Review your work before finalizing it in the repository.



1. Working Directory:

- Contains all your local files and any changes you make.
- This includes new, modified, or untracked files.

2. git add:

- Moves selected changes from the Working Directory to the Staging Area.
- Prepares files to be included in the next commit.

3. Staging Area:

- Holds files that are ready to be committed.
- Also called the index, it lets you organize changes before committing.

4. git commit:

- Saves the staged changes as a snapshot in the repository.
- Records a point in the project's history for future reference.

5. Repository:

- Stores the complete history of committed changes.
- This is your local Git repository where all snapshots are kept.

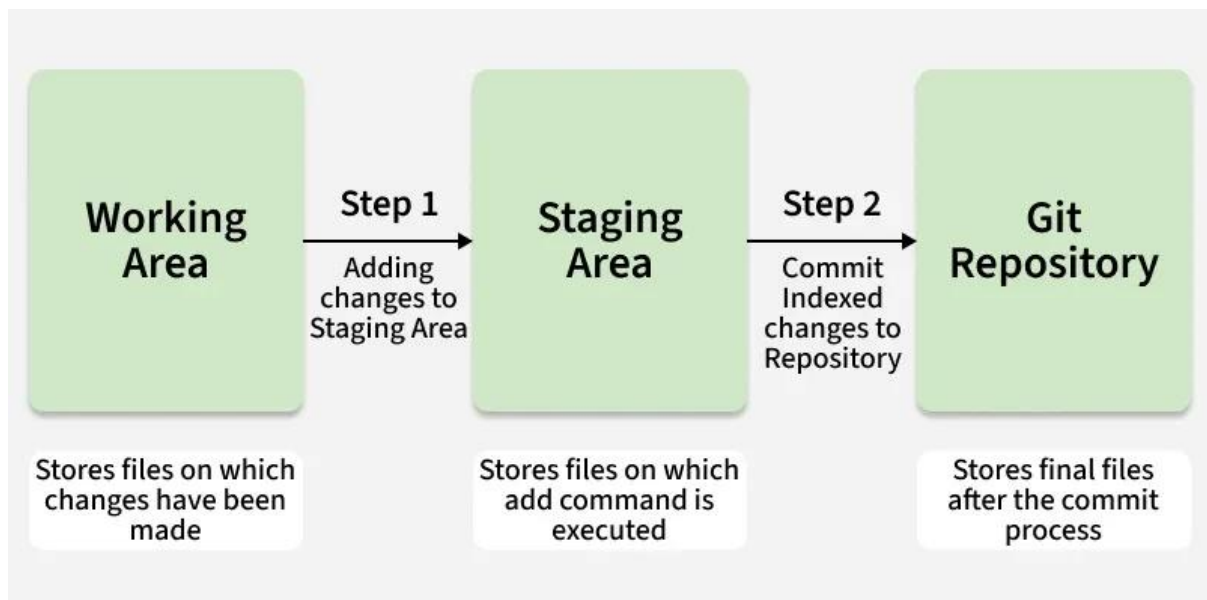
Moving From Staging Area to Commit Area:

- Add project files to the staging area using git add.
- Committing is done on files in the staging area (Index).
- Use git commit to save staged changes to the local repository.

Syntax And Usage Of `git commit`

\$ git commit -m "Add existing file"

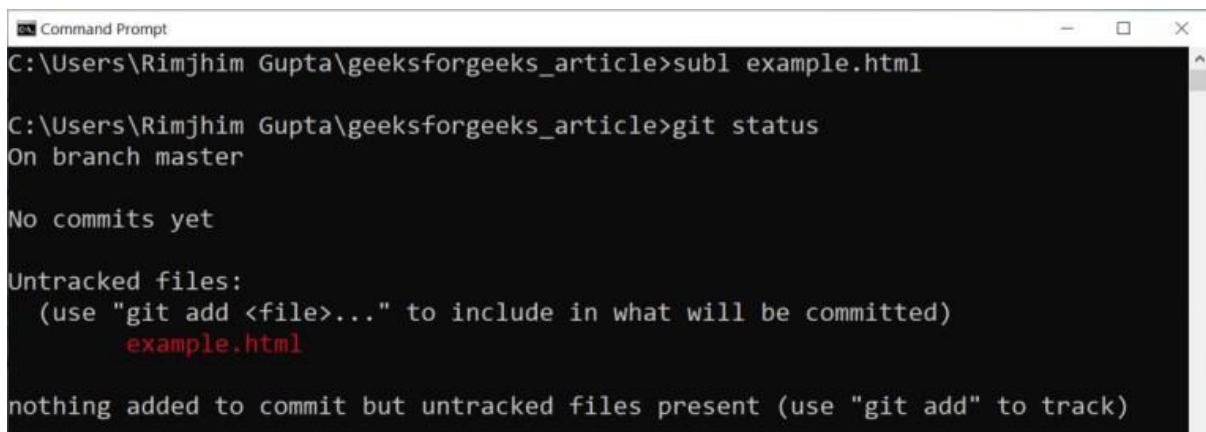
- The git commit -m command stages tracked files and commits them.
- A message is added to describe the changes for future reference.



States of File in Git Working Directory :-

1. Untracked state

- **Untracked Files:** Files not in last commit or staging area
- **New Files:** Any newly created file is untracked
- **Reason:** Git has no previous snapshot of it
- **Example:** Add example.html to repo
- **Check Status:** git status shows it as untracked



```
Command Prompt
C:\Users\Rimjhim Gupta\geeksforgeeks_article>subl example.html

C:\Users\Rimjhim Gupta\geeksforgeeks_article>git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        example.html

nothing added to commit but untracked files present (use "git add" to track)
```

Untracked State in Git

2. Tracked state

- **Tracked Files:** Files present in the last snapshot
- **Known to Git:** Already added to the repository
- **Modified:** File changed but not staged
- **Staged:** File added to staging area
- **Committed:** File saved in the repository

3. Modified State

- **Modified State:** File has changes not committed yet
- **Type of Changes:** Add, update, or delete content
- **Next Commit:** Changes will be included after staging
- **Example:** Modify example.html
- **Check Status:** Use git status

```
Command Prompt
C:\Users\Rimjhim Gupta\geeksforgeeks_article>subl example.html

C:\Users\Rimjhim Gupta\geeksforgeeks_article>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   example.html

no changes added to commit (use "git add" and/or "git commit -a")
```

Modified States in Git

4. Staging State

- **Staging State:** File prepared for next commit
- **New or Modified:** Can be untracked or modified files
- **Purpose:** Tell Git to include it in next commit
- **Command:** Use git add
- **Example:** Stage example.html
- **Check Status:** Run git status

```
Command Prompt
C:\Users\Rimjhim Gupta\geeksforgeeks_article>git add example.html

C:\Users\Rimjhim Gupta\geeksforgeeks_article>git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   example.html
```

Staging State

Now, let's stage our modified tracked example.html file and run the git status command.

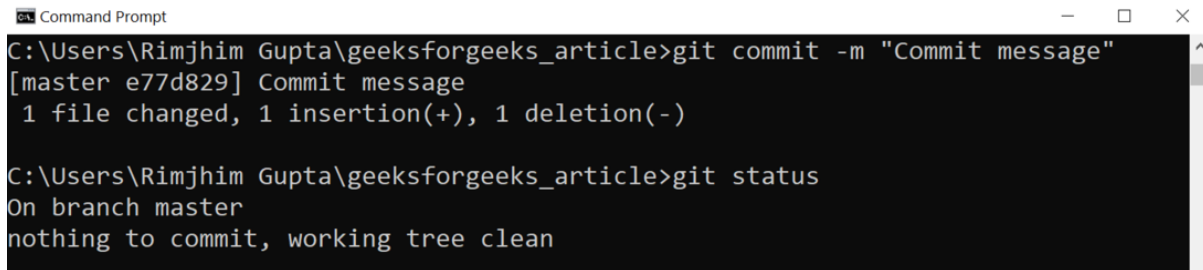
```
Command Prompt
C:\Users\Rimjhim Gupta\geeksforgeeks_article>git add example.html

C:\Users\Rimjhim Gupta\geeksforgeeks_article>git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   example.html
```

Modified Tracked file

5. Committed State

- **Committed State:** Changes saved in Git history
- **Snapshot:** Stored safely in Git directory
- **Command:** Use git commit
- **Result:** Creates a new snapshot
- **Stats:** Shows details of changes made

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt". The window shows the execution of two Git commands. The first command is "git commit -m 'Commit message'", which outputs "[master e77d829] Commit message" and "1 file changed, 1 insertion(+), 1 deletion(-)". The second command is "git status", which outputs "On branch master" and "nothing to commit, working tree clean".

```
Command Prompt
C:\Users\Rimjhim Gupta\geeksforgeeks_article>git commit -m "Commit message"
[master e77d829] Commit message
1 file changed, 1 insertion(+), 1 deletion(-)

C:\Users\Rimjhim Gupta\geeksforgeeks_article>git status
On branch master
nothing to commit, working tree clean
```

Git Commit

Basic Staging Commands :-

1. Stage All Changes:

- `git add .` or `git add -A` to stage all new, modified, and deleted files.

2. Stage Specific File:

- Edit a file (e.g., `a.txt`) and run `git status`.
- Green = staged, Red = unstaged.
- Example: Stage "hello" with `git add .`, unstaged "bhailogs" remains red.

3. Unstage a File:

- `git reset file_name` removes file from staging but keeps it in working directory.
- Example: `git reset b.txt`.

4. Stage by Hunk:

- `git add -p` stages parts of a file interactively.
- Actions: `y` = stage hunk, `n` = skip, `q` = quit, `a` = stage all remaining, `d` = skip all remaining, `e` = edit, `?` = help.

5. Interactive Add:

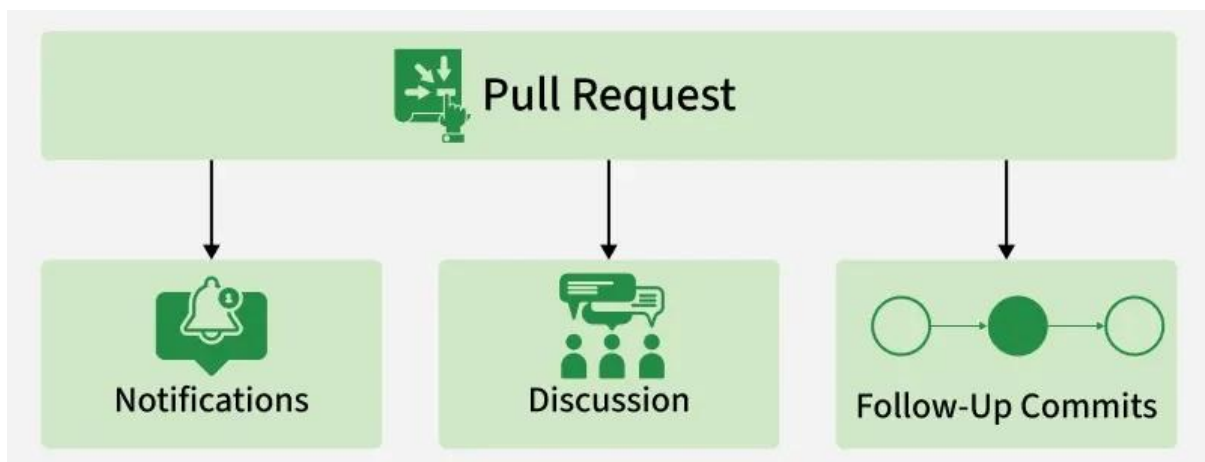
- `git add -i` for interactive staging:
- status, update, revert, add untracked, patch, diff, quit, help.

Git Pull :-

- Git pull fetches changes from the remote repository.
- It merges those changes into the local repository.
- It is a combination of git fetch and git merge.
- It updates the local branch with remote changes.
- It helps developers work with the latest code.

syntax of git pull command:

`git pull [remote-name] [branch-name]`



The image is a simple diagram explaining the components of a Pull Request.

- At the top, there is a “Pull Request” box, indicating the main action in Git/GitHub workflows.
- Three arrows point downward from the Pull Request, leading to three components:
 1. **Notifications** – Indicates that team members receive alerts when a pull request is created or updated.
 2. **Discussion** – Represents collaboration where team members can comment, review code, and provide feedback.
 3. **Follow-Up Commits** – Shows that additional commits can be made to the pull request based on feedback or changes before merging.

How Git Pull Works?

Git Pull is a command used to update the local version of a repository from a remote repository. It is a mixture of two other commands:

1. git fetch
 2. git merge
- Stage 1: git pull runs git fetch for the current branch (HEAD) to download changes.
 - Stage 2: It performs a merge, creating a new merge commit and updating HEAD.

- Example: If you pushed file.txt and a teammate added New.txt on the remote, git pull fetches and merges New.txt into your local repository.

akashadr Create New.txt

4913939 now 2 commits

New.txt	Create New.txt	now
file.txt	first commit	4 hours ago

Help people interested in this repository understand your project by adding a README.
Add a README

If you create and commit crio.txt, Git may block the push.
This happens when the remote repository has changes not present locally.

```

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git add "crio.txt"

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git status
On branch master
Your branch is up to date with 'crio/master'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   crio.txt

```

```

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git commit -m "added something new"
[master 85007c7] added something new
1 file changed, 1 insertion(+)
create mode 100644 crio.txt

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git push origin master
To https://github.com/akashadr/Git.git
 ! [rejected]        master -> master (fetch first)
error: failed to push some refs to 'https://github.com/akashadr/Git.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.

```

In this case, git pull fetches and downloads changes from the remote repository.
Running git pull origin master updates the local branch, allowing you to push your changes successfully.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git pull origin master
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Unpacking objects: 100% (3/3), 655 bytes | 3.00 KiB/s, done.
From https://github.com/akashadr/Git
 * branch      master      -> FETCH_HEAD
    5a457ea..4913939  master -> origin/master
Merge made by the 'ort' strategy.
New.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 New.txt
```

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git push origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 4 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 615 bytes | 205.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/akashadr/Git.git
    4913939..da2e9ed  master -> master
```



akashadr Merge branch 'master' of <https://github.com/akashadr/Git>

da2e9ed 21 minutes ago 4 commits

 New.txt	Create New.txt	29 minutes ago
 crio.txt	added something new	22 minutes ago
 file.txt	first commit	4 hours ago

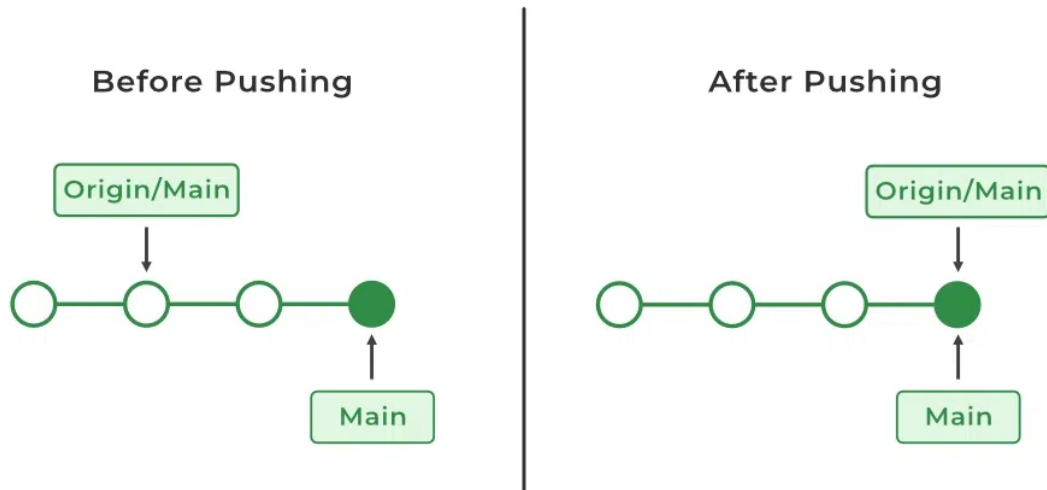
Help people interested in this repository understand your project by adding a README.

[Add a README](#)

Git Push :-

- The Git push command uploads local commits to a remote repository.
- Once pushed, other developers can access and pull these changes.
- Before pushing, you must commit your changes locally using:

git commit -m "Commit message"



- If the local branch is behind the remote, git push origin main publishes changes.

git push <remote> <branch>

1. The <remote> The alias of the remote repository (origin)
2. The <branch> The branch you want to push (main , develop)

Example:

git push origin main

Setting Up a Remote Repository :-

- If no remote is linked, add one using:

git remote add <remote-name> <remote-URL>

Example:

git remote add origin https://github.com/username/repo.git

- Now you can push your changes:

git push -u origin main

u sets origin main as the default upstream branch for future pushes.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner *

Repository name *

akashadr

 /

Crio

Great repository names are Crio is available. memorable. Need inspiration? How about [effective-waffle?](#)

Description (optional)

☒ Public

Anyone on the internet can see this repository. You choose who can commit.

☐ Private

You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☐ Add a README file

This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

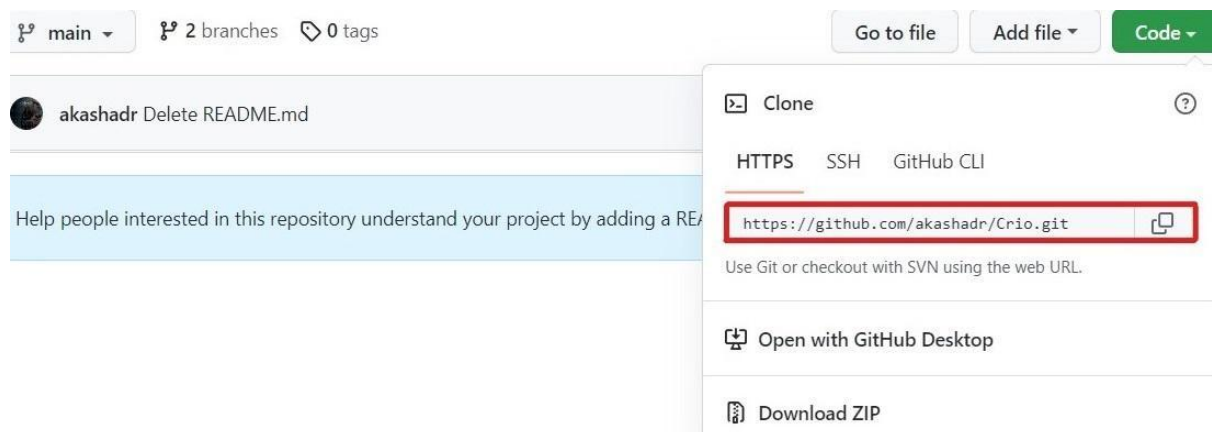
.gitignore template: None

To add this repo project to your GitHub account, use the [git remote add](#) command on the terminal, in the directory where your repository is currently stored at.

`git remote add <remote-name><remote URL>`

This command takes two arguments:

1. A remote name, let's say for example, "Origin"
2. A remote URL, which is your repo address as shown below.



Now you can run the git remote add command and git push command to push your code work to your repository.

```
Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git remote add crio "https://github.com/akashadr/Crio.git"

Akash Jha@LAPTOP-LJJ1U61G MINGW64 ~/Desktop/Git (master)
$ git push -u crio master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 209 bytes | 104.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:   https://github.com/akashadr/Crio/pull/new/master
remote:
To https://github.com/akashadr/Crio.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'crio'.
```

We used crio here in place of origin which is the repo name coz we assign an origin to another repo and if you assign an origin to a certain repo, you cannot use it again.

 master ▾

 1 branch

 0 tags

Go to file

Add file ▾

Code ▾

 akashadr first commit

5a457ea 3 hours ago

 1 commit

 file.txt

first commit

3 hours ago

Help people interested in this repository understand your project by adding a README.

Add a README

Difference Between Git & Github :-

S.No.	Git	GitHub
1.	Git is a software.	GitHub is a service.
2.	Git is a command-line tool	GitHub is a graphical user interface
3.	Git is installed locally on the system	GitHub is hosted on the web
4.	Git is maintained by linux.	GitHub is maintained by Microsoft.
5.	Git is focused on version control and code sharing.	GitHub is focused on centralized source code hosting.
6.	Git is a version control system to manage source code history.	GitHub is a hosting service for Git repositories.
7.	Git was first released in 2005.	GitHub was launched in 2008.
8.	Git has no user management feature.	GitHub has a built-in user management feature.
9.	Git is open-source licensed.	GitHub includes a free-tier and pay-for-use tier.
10.	Git has minimal external tool configuration.	GitHub has an active marketplace for tool integration.
11.	Git provides a Desktop interface named Git Gui.	GitHub provides a Desktop interface named GitHub Desktop.
12.	Git competes with CVS, Subversion, Mercurial, etc.	GitHub competes with GitLab, Bit Bucket, AWS Code Commit, Azure DevOps Server, etc.